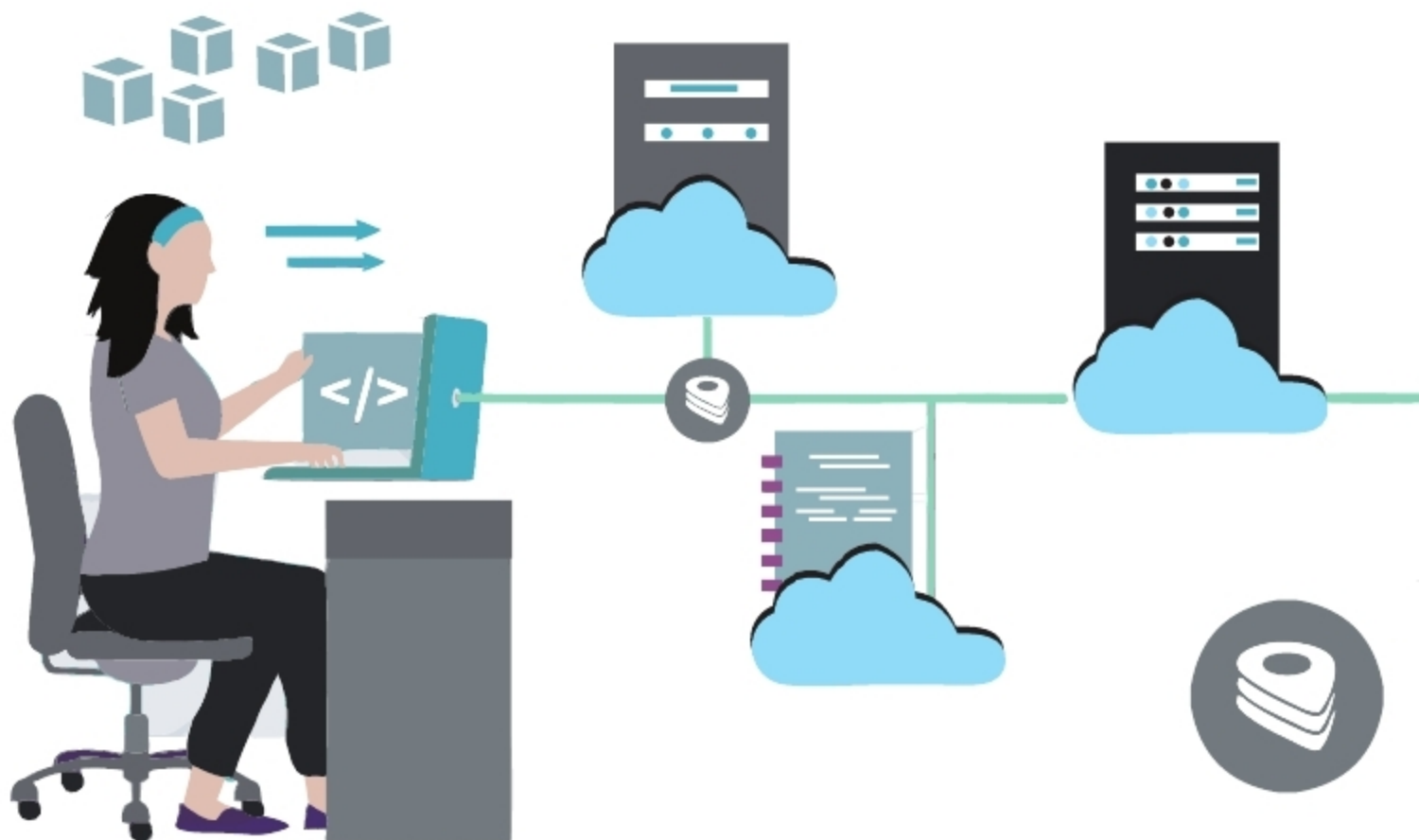


An Introduction to Local AWS Deployment with LocalStack

LocalStack provides an easy-to-use test/mock framework for developing cloud applications. It builds on the existing best-of-the-breed test tools like kinesalite/dynalite and Moto. These tools may be individually awesome but lack functionality for certain use cases. LocalStack combines these tools, makes them interoperable and adds important missing functionality on top of them.



Today, Web services are quickly moving to a cloud architecture, and DevOps and testing are following closely. While cloud architectures are highly scalable, there are some challenges to overcome before they provide the maximum benefit to businesses. There are so many benefits in running business applications such as banking services, mobile collaboration apps, mailbox on cloud apps, cloud backup apps, virtual storage drives, etc, on cloud computing environments. Yet, organisations still want to have the ability to visualise how their deployment on the cloud looks like, in order to meet any challenges, before moving on to a full-fledged cloud deployment.

Currently, with the focus primarily on supporting the AWS cloud stack, LocalStack provides an easy-to-use test/mock framework for developing cloud applications. It builds on the existing best-of-breed mock/testing tools like kinesalite/dynalite and Moto. While these tools are

individually awesome, they lack functionality for certain use cases. LocalStack combines these tools, makes them interoperable and adds important missing functionality on top of them.

Let us look at some of the functions that LocalStack adds.

Error injection: LocalStack allows injection of the errors that frequently occur in real cloud environments -- for instance, *ProvisionedThroughputExceededException*, which is thrown by Kinesis or DynamoDB if the amount of read/write throughput is exceeded.

Isolated processes: All services in LocalStack run in separate processes. The overhead of additional processes is negligible, and the entire stack can easily be executed on any developer machine and CI server. In Moto, the components are often hard-wired in the RAM, e.g., when forwarding a message on an SNS topic to an SQS queue, the queue endpoint is looked up in a local hash map. In contrast,